RELATÓRIO TÉCNICO

Implementação do controle incremental por watermark

Fase 5 — Adaptação da ETL incremental

Item	Descrição
Projeto	Data Warehouse AdventureWorks com ETL Incremental e Apache Airflow
Banco de controle	PostgreSQL — adventureworks_dw
Schema de controle	controle
Tabela	controle.etl_watermark
Atividade	Implementar controle incremental por watermark
Data do registro	27/10/2026

Status: CONCLUÍDA

1. Objetivo

A tarefa teve como objetivo implementar e validar o controle incremental da ETL por meio de watermarks persistidos no PostgreSQL. A estratégia permite identificar até qual data cada processo já foi carregado e restringir as próximas extrações aos registros novos ou modificados após a última execução bem-sucedida.

Foram validados a estrutura da tabela de controle, as funções Python de leitura e atualização, a ordem de execução no pipeline incremental e o comportamento funcional em cenários sem novos dados e com retrocesso controlado do watermark.

2. Estrutura do controle incremental

O controle está armazenado no schema controle, separado do schema dimensional dw. Essa separação evita misturar metadados operacionais da ETL com as tabelas analíticas.

Coluna	Tipo	Finalidade
processo	varchar(100) — PK	Identifica o processo ETL controlado
ultima_execucao	timestamp	Maior ModifiedDate já processado
atualizado_em	timestamptz	Data/hora em que o controle foi atualizado

Processos controlados:

- dim_produto
- dim_territorio
- fato_venda

A dim_tempo não utiliza watermark, pois é gerada deterministicamente a partir do intervalo de datas da origem e carregada com ON CONFLICT (sk_tempo) DO NOTHING.

3. Funções de leitura e atualização

O módulo etl/control/watermark.py implementa duas operações centrais: buscar_watermark() e atualizar_watermark().

Função	Operação	Comportamento
buscar_watermark()	SELECT ultima_execucao	Retorna a data registrada ou None quando não há valor
atualizar_watermark()	UPDATE controle.etl_watermark	Atualiza ultima_execucao e atualizado_em

A atualização utiliza commit após sucesso e rollback em caso de exceção. O cursor é encerrado no bloco finally.

4. Integração com o pipeline incremental

No pipeline, o watermark é consultado antes da extração. A nova data somente é calculada e persistida depois que a transformação e a carga terminam com sucesso.

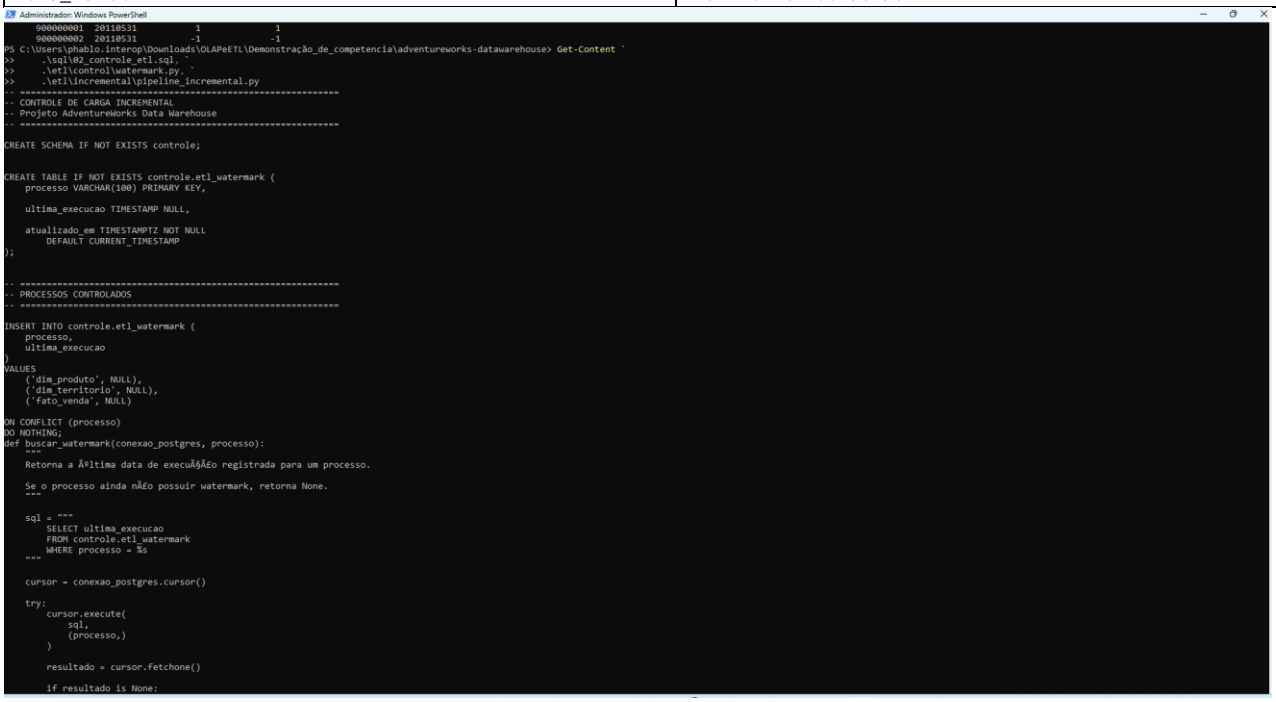
- 1. Buscar o watermark atual do processo.
- 2. Extrair apenas registros com ModifiedDate > watermark.
- 3. Transformar os registros extraídos.
- 4. Carregar no Data Warehouse.

- 5. Calcular o maior ModifiedDate processado.
- 6. Atualizar o watermark somente após a carga.

Esse desenho evita o avanço prematuro do controle. Caso ocorra falha depois do commit da carga e antes da atualização do watermark, a próxima execução poderá reprocessar registros, porém os ON CONFLICT já validados preservam a idempotência e evitam duplicidades.

5. Estado validado dos watermarks

Processo	ultima_execucao
dim_produto	2014-02-08 10:03:55.510
dim_territorio	2008-04-30 00:00:00
fato_venda	2014-07-07 00:00:00



Estrutura do controle incremental, funções de leitura/atualização e estado atual dos watermarks.

6. Teste funcional com watermarks atuais

O pipeline incremental foi executado com os watermarks já posicionados nos maiores ModifiedDate conhecidos da origem.

Processo	Extraídos	Processados	Interpretação
dim_produto	0	0	Nenhum produto novo/modificado
dim_territorio	0	0	Nenhum território novo/modificado
dim_tempo	1127	1127	Dimensão sem watermark; registros existentes são ignorados no load
fato_venda	0	0	Nenhum item de venda novo/modificado

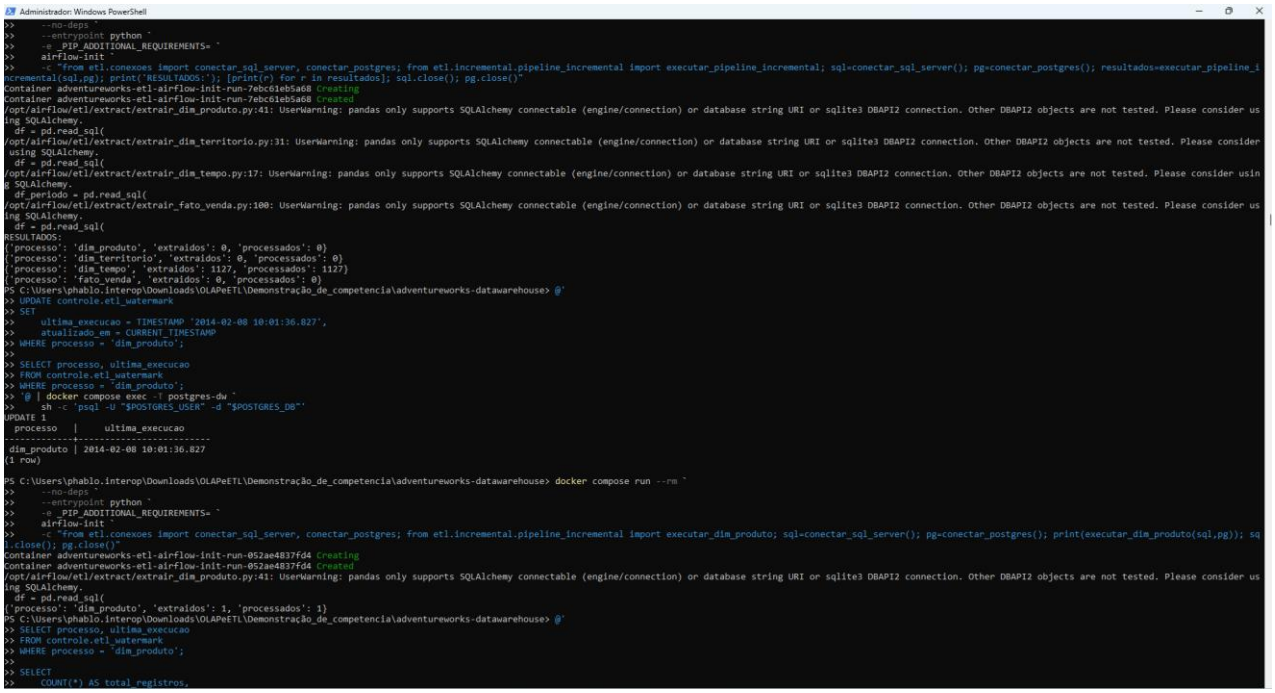
O resultado comprova que o watermark impede o reprocessamento desnecessário de produto, território e fato. Na dim_tempo, o campo processados representa registros enviados/tentados pelo loader; isso não significa 1.127 novas inserções, pois os conflitos são ignorados.

7. Teste controlado de avanço do watermark

Para comprovar que o mecanismo também permite reprocessar somente uma janela específica, o watermark de dim_produto foi temporariamente retrocedido para 2014-02-08 10:01:36.827.

Etapa	Resultado
Watermark temporário	2014-02-08 10:01:36.827
Registros extraídos	1
Registros processados	1
Watermark após a carga	2014-02-08 10:03:55.510
Total de registros em dim_produto	505
IDs distintos em dim_produto	505

O teste demonstrou que exatamente um produto posterior ao watermark foi selecionado e processado. Após a carga, o controle avançou automaticamente para o maior ModifiedDate. A dimensão permaneceu com 505 registros e 505 IDs distintos, comprovando que a carga incremental não gerou duplicidades.



Teste funcional: zero novos registros, retrocesso controlado e avanço automático do watermark sem duplicidades.

8. Observações técnicas

- Os avisos do pandas sobre conexões DBAPI2 não interromperam a execução e não afetaram os resultados dos testes.
- A métrica processados da dim_tempo representa linhas iteradas pelo loader, não necessariamente linhas fisicamente inseridas.
- O watermark é baseado em ModifiedDate da origem e utiliza comparação estrita (>), evitando reprocessar o próprio limite já persistido.

9. Resultado consolidado

Validação	Resultado
Tabela de controle persistente	Aprovada

Validação	Resultado
Leitura de watermark	Aprovada
Atualização com commit/rollback	Aprovada
Atualização somente após load	Aprovada
Zero reprocessamento com watermark máximo	Aprovado
Seleção de registro após retrocesso controlado	Aprovada
Avanço automático do watermark	Aprovado
Ausência de duplicidades após reprocessamento	Aprovada

10. Conclusão

A tarefa “Implementar controle incremental por watermark” foi concluída com sucesso. A ETL possui controle persistente por processo, consulta o último ponto processado antes da extração e atualiza o watermark somente após a conclusão da carga. Os testes mostraram que execuções sem alterações não reprocessam produto, território ou fato e que, ao retroceder o controle, somente os registros posteriores ao novo limite são selecionados.

A combinação entre watermark e cargas idempotentes com ON CONFLICT torna o processo resiliente a reexecuções e adequado às próximas etapas da Fase 5, especialmente a integração completa do pipeline incremental e os testes finais de idempotência.